T3000 API & Protocol

T3000 Mobile Architecture Guide · built Nov 13, 2026

T3000 Mobile Architecture Guide

Version: 1.0 **Date:** January 29, 2026 **Status:** ⏸️ Planning Phase - Not Started **Last Updated:** January 29, 2026

🚀 Quick Start (Resume After Break)

Haven't worked on this in a while? Start here:

1. **Check Current Status** → See Project Status below
2. **Review What's Done** → Check completed tasks in status section
3. **Find Next Task** → See unchecked items in status section
4. **Read Relevant Section** → Jump to that section number below
5. **Follow Implementation Steps** → Section 7 has detailed checklist
6. **Test** → Section 8 has testing checklist

Critical Rules (Don't Skip):

- ✅ ALWAYS test desktop after any change
- ✅ NEVER move files until mobile is working
- ✅ READ Common Pitfalls (Section 9) before coding

📊 Project Status

Current Phase: ⏸️ Phase 1 - Proof of Concept (NOT STARTED)

Completed Tasks:

- [x] Architecture planning
- [x] Documentation created
- [ ] Shared folder structure created
- [ ] Business logic extracted (Inputs)
- [ ] Mobile components created
- [ ] Device detection implemented
- [ ] View router created
- [ ] Testing completed

Next Immediate Task: → **Create folder structure** (See Section 7, Step 1)

Files Changed So Far: None (Desktop code unchanged)

Estimated Completion: Not started yet

📋 Table of Contents

1. Overview
2. Current Architecture
3. Target Architecture
4. Migration Strategy
5. Folder Structure
6. Code Patterns
7. Implementation Steps
8. Testing Checklist
9. Common Pitfalls

---

1. Overview

Goal

Add mobile-responsive UI to T3000 WebView while keeping desktop version working without changes.

Approach

Incremental Migration - Build mobile UI separately, gradually extract shared business logic, leave desktop unchanged.

Principles

- ✅ **Zero Risk to Desktop** - Desktop code stays working throughout
- ✅ **Shared Business Logic** - One source of truth for data/API calls
- ✅ **Separate UI Layers** - Desktop table vs Mobile card views
- ✅ **Gradual Migration** - One feature at a time, test thoroughly

---

2. Current Architecture

Project Structure (Before Migration)

TEXT

```
src/
├── shared/                              [Root-level shared - Vue ⇔ React]
│   ├── api/client.ts                   (Axios client - UNUSED by React)
│   ├── auth/authService.ts             (Auth service - UNUSED by React)
│   ├── state/sharedState.ts            (Cross-framework state - UNUSED)
│   └── routes.ts                       (Route detection - USED)
│
├── t3-vue/                              [Vue 3 + Quasar app]
│   └── ... (Existing Vue app - UNCHANGED)
│
└── t3-react/                            [React + FluentUI Desktop App]
    ├── app/
    │   ├── App.tsx                     (Main app component)
    │   ├── main.tsx                    (React entry point)
    │   └── router/                     (React Router setup)
    │
    ├── config/
    │   └── constants.ts                ⚠️ API_BASE_URL (51+ files depend on this)
    │
    ├── services/                        ⚠️ Core services (should be shared)
    │   ├── deviceApi.ts
    │   ├── fileMenuService.ts
    │   ├── toolsMenuService.ts
    │   └── ... (9 total)
    │
    ├── store/                           ⚠️ Global stores (should be shared)
    │   ├── authStore.ts
    │   ├── statusBarStore.ts
    │   ├── treeStore.ts
    │   └── uiStore.ts
    │
    ├── shared/                          [React-specific shared code]
    │   ├── components/                 (12 UI components)
    │   ├── hooks/                      (24 hooks)
    │   ├── services/                   (panelDataRefreshService)
    │   ├── types/                      (TypeScript types)
    │   └── utils/                      (Utilities)
    │
    ├── features/                        [27 feature modules]
    │   ├── inputs/
    │   │   ├── components/             (Desktop UI)
    │   │   ├── data/                   (rangeData.ts)
    │   │   ├── pages/
    │   │   │   └── InputsPage.tsx      (Desktop page - 1411 lines)
    │   │   └── services/
    │   │       └── inputRefreshApi.ts
```

```
│   ├── outputs/                    (Same structure)
│   ├── variables/                  (Same structure)
│   └── ... (24 more features)
│
└── layout/                          [Desktop-only layouts]
    ├── MainLayout.tsx              (TreePanel + Header + StatusBar)
    ├── Header.tsx
    ├── TreePanel.tsx
    └── StatusBar.tsx
```

Key Dependencies

`src/t3-react/config/constants.ts` imported by:

- 9 files in `services/*`
- 2 files in `shared/hooks/*`
- 1 file in `shared/components/*`
- 29 files in `features/**/pages/*`
- 8 files in `features/**/services/*`
- 1 file in `features/**/store/*`

Total: 51+ files depend on config/constants.ts

Current Import Patterns

TYPESCRIPT

```typescript
// Services import from config
import { API_BASE_URL } from '../config/constants';

// Features import from config
import { API_BASE_URL } from '../../../config/constants';

// Shared imports from config
import { API_BASE_URL } from '../../config/constants';

// Features import from shared
import { PanelDataRefreshService } from '../../../shared/services/panelDataRefreshService';
```

---

3. Target Architecture

Final Structure (After Complete Migration)

TEXT

```
src/
├── shared/                              [Platform-Agnostic Core]
│   ├── features/                       [Business Logic by Feature]
│   │   ├── inputs/
│   │   │   ├── hooks/
│   │   │   │   └── useInputsPage.ts   ← ALL page logic
│   │   │   ├── services/
│   │   │   │   ├── inputApi.ts        ← API calls
│   │   │   │   └── inputRefreshApi.ts ← Refresh logic
│   │   │   ├── stores/
│   │   │   │   └── inputStore.ts      ← State management (if needed)
│   │   │   ├── types/
│   │   │   │   └── input.types.ts     ← TypeScript interfaces
│   │   │   └── data/
│   │   │       └── rangeData.ts       ← Static data
│   │   │
│   │   ├── outputs/                   ← Same pattern
│   │   ├── variables/                 ← Same pattern
│   │   └── ... (27 features)
│   │
│   ├── core/                           [Core Infrastructure]
│   │   ├── api/
│   │   │   └── apiClient.ts           ← HTTP client
│   │   ├── config/
│   │   │   └── constants.ts           ← API_BASE_URL (from t3-react/config)
│   │   ├── services/                   ← Core services (from t3-react/services)
│   │   │   ├── deviceApi.ts
│   │   │   ├── fileMenuService.ts
│   │   │   ├── panelDataRefreshService.ts
│   │   │   └── ... (all services)
│   │   ├── stores/                     ← Global stores (from t3-react/store)
│   │   │   ├── authStore.ts
│   │   │   ├── statusBarStore.ts
│   │   │   ├── treeStore.ts
│   │   │   └── uiStore.ts
│   │   └── hooks/
│   │       ├── useDeviceType.ts       ← Detect device (desktop/mobile/tablet)
│   │       └── useBreakpoint.ts       ← Viewport detection
│   │
│   ├── components/                     [Shared UI Components]
│   │   └── ... (from t3-react/shared/components)
│   │
│   ├── hooks/                          [Shared Hooks]
│   │   └── ... (from t3-react/shared/hooks)
│   │
│   ├── types/                          [Shared Types]
```

```
│   │   └── ... (from t3-react/shared/types)
│   │
│   └── utils/                          [Utilities]
│       └── ... (from t3-react/shared/utils)
│
├── t3-react/                           [Desktop UI ONLY]
│   ├── features/
│   │   └── inputs/
│   │       ├── components/
│   │       │   ├── InputTable.tsx     ← Desktop table component
│   │       │   └── RangeSelectionDrawer.tsx
│   │       └── pages/
│   │           └── InputsPage.tsx     ← Uses shared/features/inputs/hooks
│   │
│   ├── layout/
│   │   ├── MainLayout.tsx             ← Desktop layout
│   │   ├── Header.tsx
│   │   ├── TreePanel.tsx
│   │   └── StatusBar.tsx
│   │
│   ├── components/                     ← Desktop-specific UI
│   │   └── ...
│   │
│   └── app/
│       ├── App.tsx
│       └── main.tsx
│
├── t3-mobile/                          [Mobile UI ONLY]
│   ├── features/
│   │   └── inputs/
│   │       ├── components/
│   │       │   └── InputCard.tsx      ← Mobile card component
│   │       └── pages/
│   │           └── InputsPageMobile.tsx ← Uses SAME shared hook
│   │
│   ├── layout/
│   │   ├── MobileLayout.tsx           ← Mobile layout
│   │   ├── BottomNavigation.tsx
│   │   └── MobileAppBar.tsx
│   │
│   ├── components/                     ← Mobile-specific UI
│   │   ├── MobileCard/
│   │   ├── BottomSheet/
│   │   ├── SwipeableList/
│   │   └── FloatingActionButton/
│   │
│   └── app/
│       ├── MobileApp.tsx
│       └── main.tsx
│
```

```
└── t3-vue/                          [Vue App - UNCHANGED]
    └── ...
```

New Import Patterns

TYPESCRIPT

```typescript
// Desktop page uses shared hook
import { useInputsPage } from '../../../../shared/features/inputs/hooks/useInputsPage';

// Mobile page uses SAME hook
import { useInputsPage } from '../../../../shared/features/inputs/hooks/useInputsPage';

// Both import from shared config
import { API_BASE_URL } from '../../../../shared/core/config/constants';

// Both use shared services
import { deviceApi } from '../../../../shared/core/services/deviceApi';
```

---

4. Migration Strategy

Phase 1: Proof of Concept (Week 1)

Goal: Create ONE mobile page (Inputs) without touching desktop code.

Step 1.1: Extract Business Logic

SHELL

```
## Create shared/features/inputs/
src/shared/features/inputs/hooks/useInputsPage.ts

## Extract from InputsPage.tsx:
- All useState declarations
- All API calls
- All useEffect hooks
- All event handlers
- Return all state + methods
```

Step 1.2: Create Mobile UI

SHELL

```
## Create mobile components
src/t3-mobile/components/MobileCard/MobileCard.tsx
src/t3-mobile/layout/MobileLayout.tsx

## Create mobile page
src/t3-mobile/features/inputs/pages/InputsPageMobile.tsx
```

Step 1.3: Device Detection

```shell
## Create device detection hook
src/shared/core/hooks/useDeviceType.ts

## Returns: 'desktop' | 'mobile' | 'tablet'
```

Step 1.4: Routing

```shell
## Update router to detect device
src/shared/core/router/ViewRouter.tsx

## Route to desktop or mobile version based on viewport
```

Step 1.5: Test

- ✅ Desktop InputsPage still works (unchanged)
- ✅ Mobile InputsPageMobile renders on small viewport
- ✅ Both use same data (shared hook)
- ✅ Both can fetch/save data

Phase 2: Expand Mobile Pages (Week 2-3)

Repeat Phase 1 for:

1. Outputs
2. Variables
3. Devices/Discover
4. Trendlogs
5. Schedules

Phase 3: Refactor Desktop (Optional - Week 4+)

Once mobile is working, optionally refactor desktop to use shared hooks:

```typescript
// OLD Desktop Code (before)
const InputsPage = () => {
  const [inputs, setInputs] = useState([]);
  // ... 1411 lines of logic
};

// NEW Desktop Code (after - optional)
const InputsPage = () => {
  const logic = useInputsPage(); // Use shared hook

  return <DesktopTable items={logic.inputs} />;
};
```

---

5. Folder Structure

Detailed File Layout

TEXT

```
src/shared/features/inputs/
├── hooks/
│   └── useInputsPage.ts              [ALL business logic]
├── services/
│   ├── inputApi.ts                   [CRUD operations]
│   └── inputRefreshApi.ts            [Refresh from device]
├── types/
│   └── input.types.ts                [TypeScript interfaces]
└── data/
    └── rangeData.ts                  [Static range definitions]
```

TEXT

```
src/t3-mobile/features/inputs/
├── components/
│   ├── InputCard.tsx                 [Mobile card view]
│   └── InputCard.module.css
└── pages/
    ├── InputsPageMobile.tsx          [Mobile page]
    └── InputsPageMobile.module.css
```

TEXT

```
src/t3-mobile/components/
├── MobileCard/
│   ├── MobileCard.tsx                [Generic card component]
│   └── MobileCard.module.css
├── BottomSheet/
│   ├── BottomSheet.tsx               [Modal for mobile]
│   └── BottomSheet.module.css
└── FloatingActionButton/
    ├── FloatingActionButton.tsx      [FAB for actions]
    └── FloatingActionButton.module.css
```

TEXT

```
src/shared/core/hooks/
├── useDeviceType.ts                  [Device detection]
└── useBreakpoint.ts                  [Viewport size detection]
```

---

6. Code Patterns

Pattern 1: Shared Business Logic Hook

File: `src/shared/features/inputs/hooks/useInputsPage.ts`

TYPESCRIPT

```typescript
import { useState, useEffect, useCallback, useRef } from 'react';
import { useDeviceTreeStore } from '../../../../t3-react/features/devices/store/deviceTreeStore';
import { API_BASE_URL } from '../../../core/config/constants';
import { InputPoint } from '../types/input.types';

/**
 * Shared business logic for Inputs page
 * Used by both Desktop and Mobile versions
 */
export const useInputsPage = () => {
  // ============================================
  // STATE - Extracted from InputsPage.tsx
  // ============================================
  const { selectedDevice } = useDeviceTreeStore();
  const [inputs, setInputs] = useState<InputPoint[]>([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [refreshing, setRefreshing] = useState(false);
  const [editingCell, setEditingCell] = useState<{
    id: string;
    field: string;
  } | null>(null);
  const [editValue, setEditValue] = useState('');

  // ============================================
  // METHODS - Extracted from InputsPage.tsx
  // ============================================

  /**
   * Fetch inputs from backend
   */
  const fetchInputs = useCallback(async () => {
    if (!selectedDevice) {
      setInputs([]);
      return;
    }

    setLoading(true);
    setError(null);

    try {
      const url = `${API_BASE_URL}/api/t3_device/devices/${selectedDevice.serialNumber}/input-points`;
      const response = await fetch(url);

      if (!response.ok) {
        throw new Error(`Failed to fetch inputs: ${response.statusText}`);
      }

      const data = await response.json();
      setInputs(data.input_points || []);
```

```
  } catch (err) {
    const errorMessage = err instanceof Error ? err.message : 'Failed to load inputs';
    setError(errorMessage);
    console.error('Error fetching inputs:', err);
  } finally {
    setLoading(false);
  }
}, [selectedDevice]);

/**
 * Refresh single input from device
 */
const refreshSingleInput = async (index: number) => {
  // ... refresh logic
};

/**
 * Save input value
 */
const saveInput = async (id: string, field: string, value: any) => {
  // ... save logic
};

// ============================================
// EFFECTS - Extracted from InputsPage.tsx
// ============================================

useEffect(() => {
  fetchInputs();
}, [selectedDevice, fetchInputs]);

// ============================================
// RETURN - Everything the UI needs
// ============================================

return {
  // State
  inputs,
  loading,
  error,
  refreshing,
  editingCell,
  editValue,
  selectedDevice,

  // Methods
  fetchInputs,
  refreshSingleInput,
  saveInput,
  setEditingCell,
  setEditValue,
```

```
  };
};
```

Pattern 2: Desktop Page Using Shared Hook

File: `src/t3-react/features/inputs/pages/InputsPage.tsx`

TYPESCRIPT

```typescript
import { useInputsPage } from '../../../../shared/features/inputs/hooks/useInputsPage';
import { DataGrid } from '@fluentui/react-components';

/**
 * Desktop Inputs Page - Table View
 */
export const InputsPage = () => {
  const logic = useInputsPage(); // ← Use shared hook

  if (logic.loading) {
    return <Spinner />;
  }

  return (
    <div className={styles.container}>
      <Toolbar onRefresh={logic.fetchInputs} />

      <DataGrid>
        {logic.inputs.map((input) => (
          <DataGridRow key={input.inputId}>
            <DataGridCell>{input.label}</DataGridCell>
            <DataGridCell>{input.fValue}</DataGridCell>
            {/* ... more cells */}
          </DataGridRow>
        ))}
      </DataGrid>
    </div>
  );
};
```

Pattern 3: Mobile Page Using SAME Hook

File: `src/t3-mobile/features/inputs/pages/InputsPageMobile.tsx`

TYPESCRIPT

```typescript
import { useInputsPage } from '../../../../shared/features/inputs/hooks/useInputsPage';
import { MobileCard } from '../../../components/MobileCard/MobileCard';

/**
 * Mobile Inputs Page - Card List View
 */
export const InputsPageMobile = () => {
  const logic = useInputsPage(); // ← SAME hook as desktop!

  if (logic.loading) {
    return <Spinner />;
  }

  return (
    <div className={styles.mobileContainer}>
      <MobileAppBar title="Inputs" onRefresh={logic.fetchInputs} />

      <div className={styles.cardList}>
        {logic.inputs.map((input) => (
          <MobileCard
            key={input.inputId}
            title={input.label}
            value={input.fValue}
            unit={input.units}
            status={input.status}
            onTap={() => handleCardTap(input)}
          />
        ))}
      </div>
    </div>
  );
};
```

Pattern 4: Device Detection Hook

File: `src/shared/core/hooks/useDeviceType.ts`

TYPESCRIPT

```typescript
import { useState, useEffect } from 'react';

export type DeviceType = 'desktop' | 'mobile' | 'tablet';

/**
 * Detect device type based on viewport width
 */
export const useDeviceType = (): DeviceType => {
  const [deviceType, setDeviceType] = useState<DeviceType>('desktop');

  useEffect(() => {
    const detectDevice = () => {
      const width = window.innerWidth;

      if (width < 768) {
        setDeviceType('mobile');
      } else if (width < 1024) {
        setDeviceType('tablet');
      } else {
        setDeviceType('desktop');
      }
    };

    // Detect on mount
    detectDevice();

    // Detect on resize
    window.addEventListener('resize', detectDevice);
    return () => window.removeEventListener('resize', detectDevice);
  }, []);

  return deviceType;
};
```

Pattern 5: View Router

File: `src/shared/core/router/ViewRouter.tsx`

TYPESCRIPT

```typescript
import { useDeviceType } from '../hooks/useDeviceType';
import { InputsPage } from '../../../t3-react/features/inputs/pages/InputsPage';
import { InputsPageMobile } from '../../../t3-mobile/features/inputs/pages/InputsPageMobile';

/**
 * Route to desktop or mobile version based on device
 */
export const InputsRoute = () => {
  const deviceType = useDeviceType();

  return deviceType === 'mobile'
    ? <InputsPageMobile />
    : <InputsPage />;
};
```

---

7. Implementation Steps

Step-by-Step Checklist

✅ Step 1: Create Shared Folder Structure

SHELL

```shell
mkdir -p src/shared/features/inputs/hooks
mkdir -p src/shared/features/inputs/services
mkdir -p src/shared/features/inputs/types
mkdir -p src/shared/features/inputs/data
mkdir -p src/shared/core/hooks
```

✅ Step 2: Create Mobile Folder Structure

SHELL

```shell
mkdir -p src/t3-mobile/features/inputs/components
mkdir -p src/t3-mobile/features/inputs/pages
mkdir -p src/t3-mobile/components/MobileCard
mkdir -p src/t3-mobile/layout
mkdir -p src/t3-mobile/app
```

✅ Step 3: Extract Types

SHELL

```shell
## Create src/shared/features/inputs/types/input.types.ts
## Copy InputPoint interface from InputsPage.tsx
```

✅ Step 4: Extract Business Logic

SHELL

```
## Create src/shared/features/inputs/hooks/useInputsPage.ts
## Extract from InputsPage.tsx:
1. All useState declarations
2. All API calls (fetchInputs, saveInput, etc.)
3. All useEffect hooks
4. All event handlers
5. Return object with state + methods
```

✅ Step 5: Create Device Detection

SHELL

```
## Create src/shared/core/hooks/useDeviceType.ts
## Implement viewport-based detection
```

✅ Step 6: Create Mobile Components

SHELL

```
## Create src/t3-mobile/components/MobileCard/MobileCard.tsx
## Create src/t3-mobile/layout/MobileLayout.tsx
## Create src/t3-mobile/layout/MobileAppBar.tsx
```

✅ Step 7: Create Mobile Page

SHELL

```
## Create src/t3-mobile/features/inputs/pages/InputsPageMobile.tsx
## Import useInputsPage hook
## Render card list view
```

✅ Step 8: Create View Router

SHELL

```
## Create src/shared/core/router/ViewRouter.tsx
## Detect device and route to desktop or mobile
```

✅ Step 9: Update App Router

SHELL

```
## Update src/t3-react/app/router/routes.ts
## Use ViewRouter for /inputs route
```

✅ Step 10: Test

```shell
## Desktop: Resize browser to > 1024px → Should show desktop table
## Mobile: Resize browser to < 768px → Should show mobile cards
## Both should fetch same data
## Both should be able to save changes
```

---

8. Testing Checklist

Desktop Testing (After Migration)

- [ ] Desktop page loads without errors
- [ ] Data fetches correctly
- [ ] Table displays all columns
- [ ] Sorting works
- [ ] Filtering works
- [ ] Inline editing works
- [ ] Save button works
- [ ] Refresh button works
- [ ] Device selection works
- [ ] Auto-refresh works
- [ ] No console errors
- [ ] No TypeScript errors

Mobile Testing

- [ ] Mobile page loads on small viewport
- [ ] Data fetches correctly
- [ ] Cards display all info
- [ ] Tap to expand works
- [ ] Edit modal opens
- [ ] Save works
- [ ] Refresh works (pull-to-refresh)
- [ ] Bottom navigation works
- [ ] No console errors
- [ ] No TypeScript errors

Cross-Platform Testing

- [ ] Desktop → Mobile resize works
- [ ] Mobile → Desktop resize works
- [ ] Data is consistent between views
- [ ] Changes saved on desktop appear on mobile
- [ ] Changes saved on mobile appear on desktop
- [ ] Multiple devices sync correctly

---

9. Common Pitfalls

❌ Pitfall 1: Breaking Desktop Imports

Problem:

```
TYPESCRIPT
// Moving files breaks existing imports
mv src/t3-react/config/constants.ts → src/shared/core/config/constants.ts
// Now 51+ files have broken imports!
```

Solution:

```
SHELL
## DON'T move files until mobile is working
## Instead, create NEW files and gradually migrate
## Keep old files until all imports updated
```

❌ Pitfall 2: Duplicate Logic

Problem:

```
TYPESCRIPT
// Desktop has logic
const InputsPage = () => {
  const [inputs, setInputs] = useState([]);
  // ... business logic
};

// Mobile duplicates logic
const InputsPageMobile = () => {
  const [inputs, setInputs] = useState([]);
  // ... SAME business logic (BAD!)
};
```

Solution:

TYPESCRIPT

```typescript
// Extract to shared hook FIRST
export const useInputsPage = () => {
  const [inputs, setInputs] = useState([]);
  // ... business logic
  return { inputs, ... };
};

// Both use same hook
const InputsPage = () => {
  const logic = useInputsPage();
  return <DesktopTable items={logic.inputs} />;
};

const InputsPageMobile = () => {
  const logic = useInputsPage();
  return <MobileCards items={logic.inputs} />;
};
```

❌ Pitfall 3: Hardcoded Viewport Sizes

Problem:

TYPESCRIPT

```typescript
// Hardcoded breakpoints everywhere
const isMobile = window.innerWidth < 768;
```

Solution:

TYPESCRIPT

```typescript
// Centralized hook
const deviceType = useDeviceType();
```

❌ Pitfall 4: Forgetting TypeScript Types

Problem:

TYPESCRIPT

```typescript
// Shared hook but types are in desktop code
import { useInputsPage } from '../../../../shared/features/inputs/hooks/useInputsPage';
import { InputPoint } from '../../../../t3-react/features/inputs/pages/InputsPage'; // ❌ Wrong!
```

Solution:

TYPESCRIPT

```typescript
// Types also in shared
import { useInputsPage } from '../../../../shared/features/inputs/hooks/useInputsPage';
import { InputPoint } from '../../../../shared/features/inputs/types/input.types'; // ✅ Correct
```

❌ Pitfall 5: Not Testing Desktop After Changes

Problem:

```
SHELL

## Create mobile page, test mobile, forget to test desktop
## Desktop breaks but you don't notice until later
```

Solution:

```
SHELL

## Test BOTH after every change:
1. Test desktop (resize to > 1024px)
2. Test mobile (resize to < 768px)
3. Test resize between them
```

---

10. Quick Reference

File Locations

What	Desktop	Mobile	Shared
Page Component	`t3-react/features/*/pages/*.tsx`	`t3-mobile/features/*/pages/*.tsx`	N/A
Business Logic	N/A	N/A	`shared/features/*/hooks/*.ts`
API Calls	N/A	N/A	`shared/features/*/services/*.ts`
Types	N/A	N/A	`shared/features/*/types/*.ts`
UI Components	`t3-react/features/*/components/*.tsx`	`t3-mobile/features/*/components/*.tsx`	N/A
Layout	`t3-react/layout/*.tsx`	`t3-mobile/layout/*.tsx`	N/A
Config	N/A	N/A	`shared/core/config/*.ts`
Global Services	N/A	N/A	`shared/core/services/*.ts`
Global Stores	N/A	N/A	`shared/core/stores/*.ts`

Import Patterns

```typescript
// Desktop page imports shared hook
import { useInputsPage } from '../../../../shared/features/inputs/hooks/useInputsPage';

// Mobile page imports shared hook
import { useInputsPage } from '../../../../shared/features/inputs/hooks/useInputsPage';

// Hook imports from shared services
import { API_BASE_URL } from '../../../core/config/constants';

// Hook imports from feature types
import { InputPoint } from '../types/input.types';
```

Breakpoints

Device	Width	Display
Mobile	< 768px	Card list, bottom nav, FAB
Tablet	768px - 1024px	TBD (use desktop or mobile?)
Desktop	> 1024px	Table, sidebar, toolbar

---

11. Next Steps

Phase 1 Tasks (This Week)

- [ ] Create folder structure (shared/ and t3-mobile/)
- [ ] Extract useInputsPage hook
- [ ] Create MobileCard component
- [ ] Create InputsPageMobile
- [ ] Create useDeviceType hook
- [ ] Create ViewRouter
- [ ] Update app router
- [ ] Test desktop (unchanged)
- [ ] Test mobile (new)
- [ ] Document any issues

Phase 2 Tasks (Next 2-3 Weeks)

- [ ] Repeat for Outputs
- [ ] Repeat for Variables
- [ ] Repeat for Devices
- [ ] Repeat for Trendlogs
- [ ] Create mobile navigation
- [ ] Add touch gestures
- [ ] Optimize mobile performance

Phase 3 Tasks (Optional - Later)

- [ ] Refactor desktop to use shared hooks
- [ ] Move services to shared/
- [ ] Move stores to shared/
- [ ] Consolidate duplicate code
- [ ] Add tablet-specific UI

---

12. Contact & Support

Questions? Issues?

- Check this document first
- Review code examples above
- Test on both desktop and mobile before asking

Update this document when:

- Adding new patterns
- Finding new pitfalls
- Discovering better approaches
- Completing major milestones

---

13. How to Resume After a Break

If You Haven't Worked on This in Weeks/Months:

Step 1: Check Project Status

```shell
## Open this file
docs/MOBILE_ARCHITECTURE_GUIDE.md

## Look at "Project Status" section at top
## See what's completed vs what's left
```

Step 2: Verify Desktop Still Works

```shell
## Start dev server
npm run dev

## Open http://localhost:3003/#/t3000/inputs
## Test: Can you see inputs? Can you edit? Can you save?
## If YES → Desktop is fine, continue
## If NO → Something broke, check git history
```

Step 3: Check What Exists

```shell
## Check if shared folders exist
ls src/shared/features/
ls src/t3-mobile/

## If folders exist → Continue from where you left off
## If folders don't exist → Start from Step 1 in Section 7
```

Step 4: Run Tests

```shell
## Check for TypeScript errors
npm run type-check

## Check for build errors
npm run build

## If errors → Fix them before continuing
## If no errors → Good to continue
```

Step 5: Find Your Next Task

```shell
## Look at "Project Status" section
## Find first unchecked [ ] item
## Go to that section in this document
## Follow the instructions
```

If Implementation Was Interrupted:

Desktop broke?

```shell
## Rollback changes
git status
git diff
git restore <file>

## Or reset to last working commit
git log --oneline
git reset --hard <commit-hash>
```

Mobile not working?

```shell
## Mobile is separate, won't affect desktop
## Just fix the mobile code
## Desktop should still work
```

Lost track of what you were doing?

```shell
## Check git log
git log --oneline --graph --all --decorate

## Check uncommitted changes
git status
git diff

## Read commit messages to understand progress
```

Common "Resume After Break" Scenarios:

Scenario 1: Forgot what the architecture is → Re-read Sections 2, 3, 4

Scenario 2: Forgot how to extract shared hooks → Re-read Section 6, Pattern 1

Scenario 3: Desktop is broken → Check Section 9, Pitfall 1

Scenario 4: Don't remember where files should go → Check Section 10, Quick Reference table

Scenario 5: Not sure if mobile is working → Follow Section 8, Mobile Testing checklist

---

14. Maintenance & Updates

When to Update This Document:

1. **After completing a major milestone**
   - Update "Project Status" section
   - Check off completed tasks
   - Update "Last Updated" date
2. **When you discover a new pitfall**
   - Add to Section 9 (Common Pitfalls)
   - Include solution
3. **When you find a better pattern**
   - Add to Section 6 (Code Patterns)
   - Note why it's better
4. **When folder structure changes**
   - Update Section 5 (Folder Structure)
   - Update Section 10 (Quick Reference)

Version History:

Version	Date	Changes	Author
1.0	Jan 29, 2026	Initial creation, planning phase	AI + Team

Last Updated: January 29, 2026 **Version:** 1.0 **Status:** ✅ Ready to Start Phase 1

Remember: Update the "Project Status" section at the top when you complete tasks!